1　実験の進め方

本実験では，課題を **練習課題** と **必須課題** に分けて進める．練習課題は，実験環境や操作方法に慣れ，結果の見方やソースコードの流れを理解することを目的とした課題である．まずは練習課題に取り組み，実験の基本的な手順を確認することが重要である．

これに対し，必須課題は，実験内容の理解を確認するために取り組む課題であり，レポート提出の対象となる課題である．したがって，必須課題については，単に実行するだけでなく，得られた結果を整理し，必要な考察を加えたうえでまとめることが求められる．

本資料に掲載する一部のソースコードには，実行環境や対象チェーンに応じて変更すべき箇所を*___* として示している．この *___* の部分を各自の実験条件に合わせて置き換えたうえで，ソースコードを実行すること．

練習課題

操作方法や結果の見方に慣れるための課題である．まずはこちらを実施して，ソースコードの流れを確認すること．

必須課題

レポート提出の対象となる課題である．

2　実験全体の環境構築

本実験では，WSL 上の Kali Linux 環境を用いて作業を行う．まず，Windows の管理者権限付きコマンドプロンプトで WSL を利用し，Kali Linux を導入する．導入コマンドをコマンド 1 に示す．

コマンド 1　WSL を利用した Kali Linux の導入コマンド

```
wsl --install -d kali-linux
```

VS Code から wsl と Remote SSH をインストールする．

導入後は，Kali Linux 上で必要なパッケージを更新し，Python 仮想環境および Java 実行環境などを整備する．Python 関連パッケージの導入コマンドをコマンド 2 に示す．

コマンド 2　Python 環境整備コマンド

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv
```

作業ディレクトリと Python 仮想環境の作成コマンドをコマンド 3 に示す．

コマンド 3　作業ディレクトリと Python 仮想環境の作成コマンド

```
mkdir ~/temp
cd ~/temp
python3 -m venv venv
```

```
source venv/bin/activate
git clone https://github.com/yanagihalab/sus_BC2
pip install -r requests.txt
```

2.1 Kali Linux 上での injectived 実行環境の整備

本実験を Kali Linux 上で実施するためには，Injective の CLI 兼ノードデーモンである injectived を利用可能な状態にする必要がある．本環境では，Linux x86_64 向けの事前ビルド済みバイナリを用いて導入する．

2.1.1 injectived の導入

injectived の取得と展開コマンドをコマンド 4 に示す．

コマンド 4　injectived の取得と展開コマンド

```
wget https://github.com/InjectiveFoundation/injective-core/releases/latest/download/
↪ linux-amd64.zip
unzip linux-amd64.zip
chmod +x injectived
sudo mv injectived /usr/local/bin/
```

injectived の実行時に共有ライブラリが見つからない場合は，同梱されている libwasmvm.x86_64.so をシステムのライブラリパスへ配置する．共有ライブラリの配置と動作確認コマンドをコマンド 5 に示す．

コマンド 5　共有ライブラリの配置と動作確認コマンド

```
sudo mv libwasmvm.x86_64.so /usr/lib/
sudo ldconfig
injectived version
```

動作確認結果の例を出力 6 に示す．

出力 6　injectived の動作確認結果

```
Version v1.18.3 (b18483b)
Compiled at 20260331-2102 using Go go1.23.9 (amd64)
```

2.2 wallet の作成

まず，鍵名 mykey のウォレットを作成する．ウォレット作成コマンドをコマンド 22 に示す．

コマンド 7　ウォレット作成コマンド

```
injectived keys add mykey
```

すでにウォレットを作成済みの場合は，鍵を確認し，使用するアドレスを取得する．鍵情報の確認コマンドをコマンド 8 に示す．

コマンド 8　鍵情報の確認コマンド

```
injectived keys list
injectived keys show mykey -a
```

次に，Injective testnet（テストネット）用の共通設定を環境変数として与える．環境変数の設定コマンドをコマンド 23 に示す．

コマンド 9　Injective testnet 用環境変数の設定コマンド

```
export CHAIN_ID=injective-888
export NODE=https://injective-testnet-rpc.publicnode.com:443
export GAS_PRICES=500000000inj
export GAS=1000000
export KEY_NAME=mykey
export MY_ADDR=$(injectived keys show $KEY_NAME -a)
export SUBDENOM=mytoken
```

faucet（フォーセット）実行後に，残高が反映されたかを確認するため，ウォレット残高を確認する．残高確認コマンドをコマンド 24 に示す．

コマンド 10　faucet 実行後の残高確認コマンド

```
injectived query bank balances $MY_ADDR --node=$NODE
```

出力結果に `inj` の残高が含まれていれば，faucet による入金が反映されていると判断できる．

2.3　CW20 コントラクト用の Rust / CosmWasm 環境構築

CW20 コントラクトをビルドするためには，Rust および WebAssembly 向けのビルド環境が必要である．本実験では，CosmWasm の `cw-plus` に含まれる `cw20-base` をサンプルコードとして利用する．

まず，ビルドに必要な基本パッケージを導入する．

コマンド 11　Rust / CosmWasm ビルド用パッケージの導入コマンド

```
sudo apt update
sudo apt install -y build-essential curl git pkg-config libssl-dev clang cmake
```

次に，Rust を導入する．

コマンド 12　Rust の導入コマンド

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
rustc --version
cargo --version
```

CosmWasm コントラクトは WASM 形式にビルドするため，`wasm32-unknown-unknown` ターゲットを追加する．

コマンド 13　WASM ビルドターゲットの追加コマンド

```
rustup target add wasm32-unknown-unknown
```

以上により，CW20 コントラクトをビルドするための Rust / CosmWasm 基本環境が整う．

2.3.1　Docker 環境での実行（該当者のみ）

本実験では，原則として Kali Linux 上に直接導入した `injectived` を用いる．ただし，Docker 環境で動作確認を行う場合は，コマンド 14 を用いる．

コマンド 14　Docker による injectived の確認コマンド

```
docker run -it --rm injectivelabs/injective-core:v1.14.1 injectived version
```

Docker を利用する場合の導入例をコマンド 15 に示す．

コマンド 15　Docker の導入コマンド例

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
sudo docker --version
```

2.4　SimBlock の動作環境構築

SimBlock を利用するためには，Java 実行環境が必要である．本実験では，SimBlock に付属する Gradle Wrapper を用いてビルドおよび実行を行う．セットアップコマンドをコマンド 16 に示す．

コマンド 16　SimBlock 動作環境構築コマンド

```
sudo apt update
sudo apt install -y openjdk-11-jdk unzip git
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export PATH="$JAVA_HOME/bin:$PATH"
cd ~/simblock
chmod +x gradlew
./gradlew clean
./gradlew build
./gradlew :simulator:run
```

3　Day1

3.1　Cosmos 系ブロックチェーンの分析

Cosmos は，複数のブロックチェーンが相互に連携することを目的として設計されたブロックチェーンエコシステムである [1]．従来のブロックチェーンは，それぞれが独立して動作することが多く，異なるチェーン間で資産やデータをやり取りすることは容易ではなかった．これに対し Cosmos では，ブロックチェーン同士の相互運用性を重視し，独立した複数のチェーンを接続するための仕組みが整備されている [1, 3]．

Cosmos の基盤技術の一つが Cosmos SDK である [2]．Cosmos SDK は，ブロックチェーンを構築するためのモジュール群を提供する開発基盤であり，送金，アカウント管理，ステーキング，ガバナンスなどの基本機能を組み合わせることで，独自のブロックチェーンを比較的容易に構築できる [2]．このため，開発者は一からすべてを実装するのではなく，必要な機能を選択・拡張しながら独自チェーンを設計できる [2]．

また，Cosmos では Tendermint BFT を起源とするコンセンサス技術が採用されてきた [1]．この系統のコンセンサス機構では，ブロック生成と合意形成が効率的に行われ，比較的短い時間でブロックを確定できるという特徴がある [1]．そのため，トランザクションの確定時間やファイナリティの観点からも，実験対象として扱いやすいブロックチェーン基盤である．

さらに，Cosmos エコシステムにおいて重要な役割を果たすのが IBC（Inter-Blockchain Communication）である [3, 4]．IBC は，異なるブロックチェーン間でトークンやメッセージを安全にやり取りするための仕組

みであり，Cosmos の相互運用性を支える中核技術である [3, 4]．この仕組みによって，各チェーンは独立性を保ちながらも，必要に応じて他のチェーンと連携できる [3]．

Cosmos では，このような相互接続された複数のブロックチェーン群を，しばしば「ブロックチェーンのインターネット」として捉える [1]．これは，インターネットにおいて異なる計算機ネットワーク同士が相互接続され，標準化された通信手順によって情報をやり取りする構造に対応する考え方に基づくものである．すなわち，Cosmos は単一の巨大なチェーンにすべての機能を集約するのではなく，目的ごとに設計された独立チェーンを相互接続し，全体として一つの大きなネットワークを構成することを目指している [1, 3]．IBC は，この「ブロックチェーンのインターネット」を実現するための通信基盤として位置づけられる [3, 4]．

図 1 および図 2 に，インターネットとブロックチェーンのインターネットの概念比較を示す．インターネットが複数のネットワークを相互接続することで全体として一つの通信基盤を構成しているのと同様に，Cosmos では複数の独立チェーンを接続することで，全体として一つの相互運用ネットワークを構成することを目指している [1, 3]．

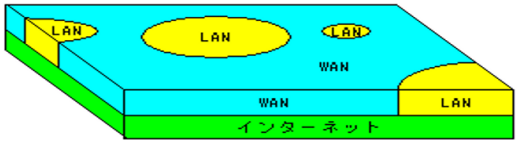

図 1　インターネット, WAN, LAN の概念的なイメージ

出典：Wikipedia「Wide Area Network」[15]

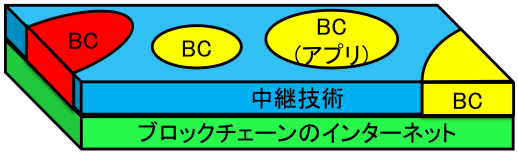

図 2　ブロックチェーンのインターネットの概念的なイメージ

本実験では，このような Cosmos の特徴を踏まえ，ブロック，トランザクション，アドレス，残高，手数料などの基本情報を確認しながら，Cosmos 系ブロックチェーンの構造と動作を理解することを目的とする．単に操作手順をなぞるのではなく，各情報がどのような意味を持ち，ブロックチェーン全体の中でどのような役割を果たしているかを意識しながら分析することが重要である．

3.2　Cosmos 系チェーンのブロックデータ取得と分析

本実験では，まず Cosmos 系チェーンのブロックデータを取得し，その後，保存済みの JSON 群に対して分析スクリプトを実行する．この作業の目的は，ブロックチェーン上に記録されている基本情報を把握し，後続のトークン実装や実験結果の解釈に必要な前提知識を得ることである．

Cosmos 系チェーンでは，各ブロックにブロック高，生成時刻，提案者アドレス，トランザクション群，署名情報などが含まれる [5]．また，各高さにおけるバリデータ集合（Validator Set）を取得することで，その時点でどのバリデータ（Validator）がネットワークの合意形成に参加していたかを確認できる [6, 5]．したがって，ブロックデータとバリデータデータをあわせて取得することで，単なるブロック列だけではなく，その背

後にある合意形成の状態も分析対象とすることができる．

3.2.1 取得対象のデータ

本実験で取得するデータは，大きく分けて次の 2 種類である．

- ブロック情報
- バリデータ情報

ブロック情報には，各高さにおけるブロックヘッダ，ブロック本文，トランザクション，署名情報などが含まれる [5]．一方，バリデータ情報には，各バリデータのアドレス，投票力（Voting Power），提案者優先度（Proposer Priority）などが含まれる [6, 5]．これらを同じ高さに対して対応付けて保存することで，後続の分析でブロック生成者とバリデータ状態の関係を調べることが可能となる．

3.2.2 データ取得の流れ

最初に，RPC ノード（Remote Procedure Call node）のエンドポイントに問い合わせて最新ブロック高を取得する．次に，その最新高さを起点として，指定したブロック数だけ過去にさかのぼりながら，各高さについてブロック情報を取得する．さらに，同じ高さ（ブロック番号）を指定してバリデータ情報を取得し，1 つの JSON ファイルにまとめて保存する．

このとき，各ファイルは 1 ブロックに対応する形で保存されるため，後から特定のブロック高だけを個別に確認したり，複数のブロックをまとめて機械的に分析したりしやすい構造となる．また，取得対象を最新から過去方向へ一定数に限定することで，大規模なブロックチェーン全体を一度に扱うのではなく，比較的新しい区間に注目した分析を行える．

3.2.3 保存形式

取得したデータは，各ブロック高ごとに JSON ファイルとして保存する．ファイル名にはブロック高を含めることで，どのファイルがどの高さに対応するかを容易に識別できるようにする．各 JSON ファイルには，少なくとも次の 2 つの要素が含まれる．

- `block_info`
- `validators`

この構造により，1 つのファイルを開くだけで，その高さにおけるブロック内容とバリデータ集合を同時に確認できる．また，後続の分析スクリプトにおいても，統一された形式でデータを読み込むことができる．

3.2.4 分析スクリプト実行の目的

取得した JSON 群に対して分析スクリプトを実行する目的は，各ブロックに含まれる情報を定量的に整理し，傾向や規則性を把握することである．ブロックデータをそのまま読むだけでは，大量の JSON から全体傾向を把握することは容易ではない．そこで，分析スクリプトを用いて必要な情報を抽出し，CSV 形式へまとめることで，後から表計算ソフトや可視化ツールを用いて二次分析しやすくする．

分析スクリプトでは，各 JSON ファイルを読み込み，たとえば次のような項目を取り出して整理する．

- ブロック高

- タイムスタンプ
- チェーン ID
- 提案者アドレス（Proposer Address）
- トランザクション数
- 署名数
- バリデータ数
- Voting Power の最大値・最小値・合計
- Proposer Priority の最大値・最小値

これらの項目をブロックごとに表形式へまとめることで，チェーンの動作を定量的に観察できるようになる．特に，Proposer Address と Proposer Priority の関係を見ることで，各ブロックの提案者（Proposer）がどのような優先度のもとで選ばれているかを確認できる．

この分析スクリプトの特徴の一つは，各ブロック単体だけでなく，直前のブロックとの関係も調べている点である．具体的には，あるブロックの Proposer Address が，前ブロックにおいて最も高い Proposer Priority を持っていたバリデータと一致するかを確認している．さらに，一致しない場合には，その Proposer が前ブロック時点で何位の Proposer Priority を持っていたかを記録している．

このような分析により，単純に「提案者が誰であったか」を記録するだけでなく，提案者選出の傾向や Priority の推移に関する観察が可能となる．その結果，ブロック提案者の選出規則や，実際のチェーン運用における偏りの有無を考察しやすくなる．

分析結果は最終的に CSV ファイルとして保存する．CSV には，ブロックごとの主要な指標が 1 行ずつ記録されるため，後からソート，集計，可視化を行うことが容易である．また，分析スクリプトでは，大きなネスト構造を持つ元の `validators` 配列などは削除したうえで保存するため，CSV として扱いやすい列構造に整理される．

以上のように，本作業は単にブロックデータを収集するだけでなく，それを分析可能な形式へ変換し，Cosmos 系チェーンの構造と動作を理解するための基盤を整えるものである．これにより，後続の CW20 実装や Injective 上での実験に進む前に，ブロックチェーン上のデータがどのように構成され，どのような観点で解析できるかを具体的に把握できる．したがって，本節は後続実験の準備段階であると同時に，Cosmos 系ブロックチェーンのデータ理解そのものを目的とする重要な工程である．

3.3 ブロック取得スクリプト

Cosmos 系チェーンのブロック情報およびバリデータ情報を取得するためのスクリプトをソースコード 17 に示す．このスクリプトでは，まず RPC から最新ブロック高を取得し，その後，指定したブロック数だけ過去にさかのぼりながら，各高さについてブロック情報とバリデータ情報を取得する．RPC のエンドポイント (BASE_URL) は以下から共同実験者と重複しないよう選択し，実行すること．

- **Osmosis**: `https://osmosis-rpc.publicnode.com:443` [必須選択]
- **Cosmos Hub**: `https://cosmos-rpc.publicnode.com:443`
- **Terra**: `https://terra-rpc.publicnode.com:443`
- **Juno**: `https://juno-rpc.publicnode.com:443`
- **Cronos**: `https://cronos-rpc.publicnode.com:443`

- **Evmos**: `https://evmos-rpc.publicnode.com:443`
- **Injective**: `https://injective-rpc.publicnode.com:443`

取得したデータは，高さごとに JSON ファイルとして保存する．これにより，後続の分析スクリプトが統一形式でデータを読み込めるようになる．

ソースコード 17　ブロック取得スクリプト（1-1）

```
import os
import requests
import json
import time
import re
from tqdm import tqdm
from urllib.parse import urlparse

BASE_URL = "*__*"

BASE_URL_BLOCK = *__* + "/block"
BASE_URL_BLOCK_RESULTS = *__* + "/block_results"
BASE_URL_VALIDATORS = *__* + "/validators"

PER_PAGE = 100
TOTAL_PAGES = 1
RETRY_LIMIT = 100
SLEEP_TIME = 1
BLOCK_COUNT = *__*

parsed_base = urlparse(BASE_URL)
hostname = parsed_base.hostname or ""

m = re.match(r"^([a-zA-Z0-9\-]+)-rpc", hostname)
BASE_NAME = m.group(1) if m else hostname.split(".")[0]

SAVE_DIR = *__*

os.makedirs(SAVE_DIR, exist_ok=True)

headers = {"User-Agent": "Mozilla/5.0"}


def get_with_retry(url, timeout=10, retry_limit=RETRY_LIMIT, sleep_time=SLEEP_TIME):
    retries = 0
    while retries < retry_limit:
        try:
            response = requests.get(url, headers=headers, timeout=timeout)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            retries += 1
            if retries >= retry_limit:
                raise e
            time.sleep(sleep_time)


def get_latest_height():
    latest_block = get_with_retry(BASE_URL_BLOCK, timeout=10)
```

```
    return int(latest_block["result"]["block"]["header"]["height"])


latest_height = *__*
print(f最新のブロック番号": *__*")
print(f"BASE_NAME: *__*")
print(f保存先ディレクトリ": *__*")

for i in tqdm(range(BLOCK_COUNT), desc="Fetching blocks", unit="block"):
    height = latest_height - i

    block_header = {}
    try:
        block_url = f"{BASE_URL_BLOCK}?height={height}"
        block_data = get_with_retry(block_url, timeout=10)
        block_header = (
            block_data.get("result", {})
            .get("block", {})
            .get("header", {})
        )
    except requests.exceptions.RequestException as e:
        print(f"  Failed to fetch block header for height {height}: {e}")

    block_results = {}
    try:
        block_results_url = f"{BASE_URL_BLOCK_RESULTS}?height={height}"
        block_results_data = get_with_retry(block_results_url, timeout=10)
        block_results = block_results_data.get("result", {})
    except requests.exceptions.RequestException as e:
        print(f"  Failed to fetch block results for height {height}: {e}")

    block_validators = []

    for page in range(1, TOTAL_PAGES + 1):
        url = f"{BASE_URL_VALIDATORS}?height={height}&per_page={PER_PAGE}&page={page}"
        retries = 0

        while retries < RETRY_LIMIT:
            try:
                response = requests.get(url, headers=headers, timeout=10)
                response.raise_for_status()

                data = *__*.json()
                result = data.get("result")

                if not result or not isinstance(result, dict) or "validators" not in result:
                    break

                validators = result["validators"]

                if not validators:
                    break

                block_validators.extend(validators)
                break

```

```
            except requests.exceptions.RequestException:
                retries += 1
                time.sleep(SLEEP_TIME)

        if retries == RETRY_LIMIT:
            print(f"  Failed to fetch validator page {page} after {RETRY_LIMIT} attempts. Skipping.")

    if block_header or block_results or block_validators:
        output = {
            "base_url": *__*,
            "blockheader": *__*,
            "blockresults": *__*,
            "validators": *__*
        }
        filename = os.path.join(SAVE_DIR, f"{BASE_NAME}_BlockNum_{height}.json")
        with open(filename, "w", encoding="utf-8") as f:
            json.dump(output, f, indent=4, ensure_ascii=False)
```

このスクリプトにおいては，RPC エンドポイント，取得ブロック数，保存先ディレクトリなどを実験目的に応じて設定する必要がある．特に BASE_URL_BLOCK，BASE_URL_VALIDATORS，BLOCK_COUNT は，各自の対象チェーンや取得範囲に応じて適切に変更すること．

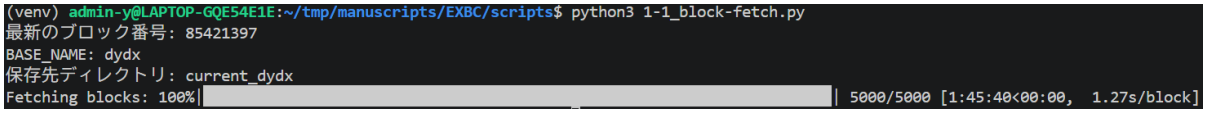

図 3　ブロックデータ取得処理の結果

Day1: 必須課題 1-1

ブロックデータを取得するソースコードの動作を説明し，取得時間や取得データサイズについて考察し，報告せよ．

3.4　ブロックチェーン分析スクリプト

保存済みの JSON 群を対象として，各ブロックの基本情報，バリデータ数，Voting Power，Proposer Priority などを抽出し，CSV 形式で整理する分析スクリプトをソースコード 18 に示す．このスクリプトでは，各 JSON ファイルを 1 ブロック単位で解析し，さらに前ブロックとの比較によって，Proposer Address と Proposer Priority の関係も調べる．最終的には，集約結果を block_analysis.csv として保存する．

ソースコード 18　ブロック分析スクリプト（1-2）

```
import os
import requests
import json
import time
import re
from tqdm import tqdm
from urllib.parse import urlparse
```

```

BASE_URL = "*__*"

BASE_URL_BLOCK = *__* + "/block"
BASE_URL_BLOCK_RESULTS = *__* + "/block_results"
BASE_URL_VALIDATORS = *__* + "/validators"

PER_PAGE = 100
TOTAL_PAGES = 1
RETRY_LIMIT = 100
SLEEP_TIME = 1
BLOCK_COUNT = *__*

parsed_base = urlparse(BASE_URL)
hostname = parsed_base.hostname or ""

m = re.match(r"^([a-zA-Z0-9\-]+)-rpc", hostname)
BASE_NAME = m.group(1) if m else hostname.split(".")[0]

SAVE_DIR = *__*

os.makedirs(SAVE_DIR, exist_ok=True)

headers = {"User-Agent": "Mozilla/5.0"}


def get_with_retry(url, timeout=10, retry_limit=RETRY_LIMIT, sleep_time=SLEEP_TIME):
    retries = 0

    while retries < retry_limit:
        try:
            response = requests.get(url, headers=headers, timeout=timeout)
            response.raise_for_status()
            return response.json()

        except requests.exceptions.RequestException as e:
            retries += 1

            if retries >= retry_limit:
                raise e

            time.sleep(sleep_time)


def get_latest_height():
    latest_block = get_with_retry(BASE_URL_BLOCK, timeout=10)

    return int(
        latest_block["result"]["block"]["header"]["height"]
    )


latest_height = *__*

print(f最新のブロック番号": *__*")
print(f"BASE_NAME: *__*")
print(f保存先ディレクトリ": *__*")
```

```python

for i in tqdm(range(BLOCK_COUNT), desc="Fetching blocks", unit="block"):
    height = latest_height - i

    block_header = {}
    block_last_commit = {}

    try:
        block_url = f"{BASE_URL_BLOCK}?height={height}"
        block_data = get_with_retry(block_url, timeout=10)

        block = (
            block_data.get("result", {})
            .get("block", {})
        )

        block_header = block.get("header", {})
        block_last_commit = block.get("last_commit", {})

    except requests.exceptions.RequestException as e:
        print(f"  Failed to fetch block header/last_commit for height {height}: {e}")

    block_results = {}

    try:
        block_results_url = f"{BASE_URL_BLOCK_RESULTS}?height={height}"
        block_results_data = get_with_retry(block_results_url, timeout=10)
        block_results = block_results_data.get("result", {})

    except requests.exceptions.RequestException as e:
        print(f"  Failed to fetch block results for height {height}: {e}")

    block_validators = []

    for page in range(1, TOTAL_PAGES + 1):
        url = f"{BASE_URL_VALIDATORS}?height={height}&per_page={PER_PAGE}&page={page}"
        retries = 0

        while retries < RETRY_LIMIT:
            try:
                response = requests.get(url, headers=headers, timeout=10)
                response.raise_for_status()

                data = *__*
                result = data.get("result")

                if not result or not isinstance(result, dict) or "validators" not in result:
                    break

                validators = result["validators"]

                if not validators:
                    break

                block_validators.extend(validators)
                break
```

```

            except requests.exceptions.RequestException:
                retries += 1
                time.sleep(SLEEP_TIME)

        if retries == RETRY_LIMIT:
            print(f"  Failed to fetch validator page {page} after {RETRY_LIMIT} attempts
↪ . Skipping.")

    if block_header or block_last_commit or block_results or block_validators:
        output = {
            "base_url": *__*,
            "blockheader": *__*,
            "last_commit": block_last_commit,
            "blockresults": *__*,
            "validators": *__*
        }

        filename = os.path.join(
            SAVE_DIR,
            f"{BASE_NAME}_BlockNum_{height}.json"
        )

        with open(filename, "w", encoding="utf-8") as f:
            json.dump(output, f, indent=4, ensure_ascii=False)
```

この分析スクリプトにより，個々のブロックに含まれる情報を表形式に整理できるため，後続の統計処理や可視化が容易になる．また，前ブロックの Proposer Priority と現ブロックの Proposer Address を比較することで，提案者選出の傾向に関する基礎的な観察も可能となる．

Day1: 必須課題 1-2

1-1 で取得したブロックデータを用いて，ソースコード 1-2 を実行せよ．またソースコードの動作を説明し，取得時間や取得データサイズについて考察し分析結果を確認したうえで考察し，報告せよ．

3.5 バリデータ分析スクリプト

バリデータ（Validator）とは，ブロックチェーンの合意形成に参加し，ブロック提案や署名を行うノードである．Cosmos / CometBFT 系チェーンでは，各ブロック高におけるバリデータ集合，Voting Power，Proposer Priority などを確認することで，合意形成に関わるノード群の変化を分析できる．

本節では，保存済み JSON 群を対象として，バリデータ集合の出現状況，Voting Power の推移，ブロック高ごとのバリデータ数，および連続ブロック間の差分を分析するスクリプトをソースコード 19 に示す．

ソースコード 19　バリデータ分析スクリプト（1-3）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import re
import json
import csv
```

```
from collections import defaultdict

import matplotlib.pyplot as *__*
from matplotlib.ticker import ScalarFormatter

INPUT_DIR = "current_*__*"
OUT_DIR = "out_*__*_validator_analysis"

PROPOSER_PRIORITY_PLAIN_FIG_WIDTH = 80
PROPOSER_PRIORITY_PLAIN_FIG_HEIGHT = 8


def extract_height_from_filename(filename: str):
    m = re.search(r"BlockNum_(\d+)\.json$", filename)
    if not m:
        return None
    return int(m.group(1))


def safe_int(value, default=0):
    try:
        return int(value)
    except Exception:
        return default


def load_validator_files(input_dir: str):
    rows = []

    for name in os.listdir(input_dir):
        if not name.endswith(".json"):
            continue

        path = os.path.join(input_dir, name)
        height = extract_height_from_filename(name)

        if height is None:
            continue

        try:
            with open(path, "*__*", encoding="utf-8") as f:
                data = json.load(*__*)

            *__*.append((height, path, data))

        except Exception as e:
            print(f"[WARN] failed to read {path}: {e}")

    rows.sort(key=lambda x: x[0])
    return rows


def normalize_validators(*__*):
    out = {}

    for v in validators:
        address = v.get("address", "")
```

```python

        if not address:
            continue

        pub_key = v.get("pub_key", {})
        pub_key_type = pub_key.get("type", "") if isinstance(pub_key, dict) else ""
        pub_key_value = pub_key.get("value", "") if isinstance(pub_key, dict) else ""

        out[address] = {
            "address": address,
            "pub_key_type": pub_key_type,
            "pub_key_value": pub_key_value,
            "voting_power": safe_int(v.get("voting_power", 0)),
            "proposer_priority": safe_int(v.get("proposer_priority", 0)),
        }

    return out


def summarize_validator_presence(*__*):
    stats = defaultdict(lambda: {
        "first_height": None,
        "last_height": None,
        "appearance_count": 0,
        "min_voting_power": None,
        "max_voting_power": None,
        "sum_voting_power": 0,
        "pub_key_type": "",
        "pub_key_value": "",
    })

    for height, _, data in blocks:
        validators = normalize_validators(data.get("validators", []))

        for addr, info in validators.items():
            st = stats[addr]

            if st["first_height"] is None:
                st["first_height"] = height

            st["last_height"] = height
            st["appearance_count"] += 1

            vp = info["voting_power"]

            if st["min_voting_power"] is None or vp < st["min_voting_power"]:
                st["min_voting_power"] = vp

            if st["max_voting_power"] is None or vp > st["max_voting_power"]:
                st["max_voting_power"] = vp

            st["sum_voting_power"] += vp
            st["pub_key_type"] = info["pub_key_type"]
            st["pub_key_value"] = info["pub_key_value"]

    return stats

```

```python

def analyze_per_height(blocks):
    per_height = []

    for height, path, data in blocks:
        validators = normalize_validators(data.get("validators", []))
        total_voting_power = sum(v["voting_power"] for v in validators.values())

        per_height.append({
            "height": height,
            "file": os.path.basename(path),
            "validator_count": len(validators),
            "total_voting_power": total_voting_power,
        })

    return per_height


def analyze_diffs(blocks):
    diffs = []

    prev_height = None
    prev_validators = None

    for height, _, data in blocks:
        current_validators = normalize_validators(data.get("validators", []))

        if prev_validators is not None:
            prev_set = set(prev_validators.keys())
            curr_set = set(current_validators.keys())

            added = sorted(curr_set - prev_set)
            removed = sorted(prev_set - curr_set)
            common = sorted(prev_set & curr_set)

            power_changed = []
            priority_changed = []

            for addr in common:
                prev_v = prev_validators[addr]
                curr_v = current_validators[addr]

                if prev_v["voting_power"] != curr_v["voting_power"]:
                    power_changed.append({
                        "address": addr,
                        "prev_voting_power": prev_v["voting_power"],
                        "curr_voting_power": curr_v["voting_power"],
                    })

                if prev_v["proposer_priority"] != curr_v["proposer_priority"]:
                    priority_changed.append({
                        "address": addr,
                        "prev_proposer_priority": prev_v["proposer_priority"],
                        "curr_proposer_priority": curr_v["proposer_priority"],
                    })

            diffs.append({
```

```
                "prev_height": prev_height,
                "curr_height": height,
                "added_count": len(added),
                "removed_count": len(removed),
                "power_changed_count": len(power_changed),
                "priority_changed_count": len(priority_changed),
                "added": added,
                "removed": removed,
                "power_changed": power_changed,
                "priority_changed": priority_changed,
            })

        prev_height = height
        prev_validators = current_validators

    return diffs


def build_validator_power_timeseries(blocks):
    heights = [height for height, _, _ in blocks]
    address_to_series = defaultdict(list)

    all_addresses = set()
    normalized_by_height = []

    for _, _, data in blocks:
        validators = normalize_validators(data.get("validators", []))
        normalized_by_height.append(validators)
        all_addresses.update(validators.keys())

    all_addresses = sorted(all_addresses)

    for addr in all_addresses:
        address_to_series[addr] = []

    for validators in normalized_by_height:
        current_addresses = set(validators.keys())

        for addr in all_addresses:
            if addr in current_addresses:
                address_to_series[addr].append(validators[addr]["voting_power"])
            else:
                address_to_series[addr].append(None)

    return heights, address_to_series


def build_validator_priority_timeseries(blocks):
    heights = [height for height, _, _ in blocks]
    address_to_series = defaultdict(list)

    all_addresses = set()
    normalized_by_height = []

    for _, _, data in blocks:
        validators = normalize_validators(data.get("validators", []))
        normalized_by_height.append(validators)
```

```
        all_addresses.update(validators.keys())

    all_addresses = sorted(all_addresses)

    for addr in all_addresses:
        address_to_series[addr] = []

    for validators in normalized_by_height:
        current_addresses = set(validators.keys())

        for addr in all_addresses:
            if addr in current_addresses:
                address_to_series[addr].append(validators[addr]["proposer_priority"])
            else:
                address_to_series[addr].append(None)

    return heights, address_to_series


def write_presence_csv(stats, out_csv):
    with open(out_csv, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)

        writer.writerow([
            "address",
            "first_height",
            "last_height",
            "appearance_count",
            "avg_voting_power",
            "min_voting_power",
            "max_voting_power",
            "pub_key_type",
            "pub_key_value",
        ])

        for addr, st in sorted(stats.items(), key=lambda x: (-x[1]["appearance_count"], x[0])):
            avg_vp = st["sum_voting_power"] / st["appearance_count"] if st["appearance_count"] else 0

            writer.writerow([
                addr,
                st["first_height"],
                st["last_height"],
                st["appearance_count"],
                f"{avg_vp:.2f}",
                st["min_voting_power"],
                st["max_voting_power"],
                st["pub_key_type"],
                st["pub_key_value"],
            ])


def write_per_height_csv(per_height, *__*):
    with open(out_csv, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)

```

```
        writer.writerow([
            "height",
            "file",
            "validator_count",
            "total_voting_power",
        ])

        for row in per_height:
            writer.writerow([
                row["height"],
                row["file"],
                row["validator_count"],
                row["total_voting_power"],
            ])


def write_diff_summary_csv(diffs, out_csv):
    with open(out_csv, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)

        writer.writerow([
            "prev_height",
            "curr_height",
            "added_count",
            "removed_count",
            "power_changed_count",
            "priority_changed_count",
        ])

        for d in diffs:
            writer.writerow([
                d["prev_height"],
                d["curr_height"],
                d["added_count"],
                d["removed_count"],
                d["power_changed_count"],
                d["priority_changed_count"],
            ])


def write_diff_details_json(diffs, out_json):
    with open(out_json, "w", encoding="utf-8") as f:
        json.dump(diffs, f, indent=2, ensure_ascii=False)


def shorten_address(addr: str, head=8, tail=6):
    if len(addr) <= head + tail + 3:
        return addr
    return f"{addr[:head]}...{addr[-tail:]}"


def apply_axis_format(use_scientific: bool, axis="both"):
    ax = plt.gca()

    if use_scientific:
        formatter_x = ScalarFormatter(useMathText=False)
        formatter_y = ScalarFormatter(useMathText=False)
```

```

        formatter_x.set_scientific(True)
        formatter_y.set_scientific(True)

        formatter_x.set_powerlimits((0, 0))
        formatter_y.set_powerlimits((0, 0))

        if axis in ("x", "both"):
            ax.xaxis.set_major_formatter(formatter_x)
            ax.ticklabel_format(axis="x", style="sci", scilimits=(0, 0))

        if axis in ("y", "both"):
            ax.yaxis.set_major_formatter(formatter_y)
            ax.ticklabel_format(axis="y", style="sci", scilimits=(0, 0))

    else:
        formatter_x = ScalarFormatter(useOffset=False)
        formatter_y = ScalarFormatter(useOffset=False)

        formatter_x.set_scientific(False)
        formatter_y.set_scientific(False)

        if axis in ("x", "both"):
            ax.xaxis.set_major_formatter(formatter_x)
            ax.ticklabel_format(axis="x", style="plain", useOffset=False)

        if axis in ("y", "both"):
            ax.yaxis.set_major_formatter(formatter_y)
            ax.ticklabel_format(axis="y", style="plain", useOffset=False)


def extract_proposer_maps(blocks):
    *__* = {}
    *__* = {}

    for height, _, data in blocks:
        blockheader = data.get("blockheader", {})
        proposer_address = blockheader.get("proposer_address", "")
        proposer_by_height[height] = proposer_address
        validators_by_height[height] = normalize_validators(data.get("validators", []))

    return proposer_by_height, validators_by_height


def plot_validator_count(per_height, out_path, use_scientific=False):
    heights = [x["height"] for x in per_height]
    counts = [x["validator_count"] for x in per_height]

    plt.figure(figsize=(12, 6))
    plt.plot(heights, counts, marker="o", markersize=2, linewidth=1)
    plt.xlabel("Height")
    plt.ylabel("Validator Count")
    plt.title("Validator Count over Height")
    plt.grid(True)
    apply_axis_format(use_scientific, axis="x")
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
```

```
    plt.close()


def plot_validator_set_changes(diffs, out_path, use_scientific=False):
    if not diffs:
        return

    heights = [x["curr_height"] for x in diffs]
    added = [x["added_count"] for x in diffs]
    removed = [x["removed_count"] for x in diffs]
    power_changed = [x["power_changed_count"] for x in diffs]

    plt.figure(figsize=(12, 6))
    plt.plot(heights, added, label="added_count", marker="o", markersize=2, linewidth=1)
    plt.plot(heights, removed, label="removed_count", marker="o", markersize=2, linewidth=1)
    plt.plot(heights, power_changed, label="power_changed_count", marker="o", markersize=2, linewidth=1)
    plt.xlabel("Current Height")
    plt.ylabel("Count")
    plt.title("Validator Set Changes over Height")
    plt.grid(True)
    plt.legend()
    apply_axis_format(use_scientific, axis="x")
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()


def plot_priority_changed_count(diffs, out_path, use_scientific=False):
    if not diffs:
        return

    heights = [x["curr_height"] for x in diffs]
    priority_changed = [x["priority_changed_count"] for x in diffs]

    plt.figure(figsize=(12, 6))
    plt.plot(heights, priority_changed, marker="o", markersize=2, linewidth=1)
    plt.xlabel("Current Height")
    plt.ylabel("Priority Changed Count")
    plt.title("Proposer Priority Changes over Height")
    plt.grid(True)
    apply_axis_format(use_scientific, axis="x")
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()


def plot_all_validators_voting_power(
    blocks,
    stats,
    out_path,
    use_scientific=True,
    fig_width=16,
    fig_height=8
):
    heights, series = build_validator_power_timeseries(blocks)
```

```python

    ranked_addresses = [
        addr for addr, _ in sorted(
            stats.items(),
            key=lambda x: (-x[1]["appearance_count"], x[0])
        )
    ]

    if not ranked_addresses:
        return

    proposer_by_height, validators_by_height = extract_proposer_maps(blocks)

    plt.figure(figsize=(fig_width, fig_height))

    line_color_by_addr = {}

    for addr in ranked_addresses:
        values = series[addr]
        line, = plt.plot(
            heights,
            values,
            label=shorten_address(addr),
            linewidth=1
        )
        line_color_by_addr[addr] = line.get_color()

    for addr in ranked_addresses:
        marker_heights = []
        marker_values = []

        for height in heights:
            if proposer_by_height.get(height, "") != addr:
                continue

            validators = validators_by_height.get(height, {})
            if addr not in validators:
                continue

            marker_heights.append(height)
            marker_values.append(validators[addr]["voting_power"])

        if marker_heights:
            plt.scatter(
                marker_heights,
                marker_values,
                marker="x",
                s=35,
                linewidths=1.2,
                color=line_color_by_addr.get(addr),
                label=None,
                zorder=5
            )

    plt.xlabel("Height")
    plt.ylabel("Voting Power")
    plt.title("Voting Power over Height")
```

```
    plt.grid(True)
    plt.legend(
        fontsize=6,
        loc="center left",
        bbox_to_anchor=(1.02, 0.5),
        borderaxespad=0.0,
        ncol=1
    )
    apply_axis_format(use_scientific, axis="both")
    plt.tight_layout(rect=[0, 0, 0.82, 1])
    plt.savefig(out_path, dpi=200, bbox_inches="tight")
    plt.close()


def plot_all_validators_proposer_priority(
    blocks,
    stats,
    out_path,
    use_scientific=True,
    fig_width=16,
    fig_height=8
):
    heights, series = build_validator_priority_timeseries(blocks)

    ranked_addresses = [
        addr for addr, _ in sorted(
            stats.items(),
            key=lambda x: (-x[1]["appearance_count"], x[0])
        )
    ]

    if not ranked_addresses:
        return

    proposer_by_height, validators_by_height = extract_proposer_maps(blocks)

    plt.figure(figsize=(fig_width, fig_height))

    line_color_by_addr = {}

    for addr in ranked_addresses:
        values = series[addr]
        line, = plt.plot(
            heights,
            values,
            label=shorten_address(addr),
            linewidth=1
        )
        line_color_by_addr[addr] = line.get_color()

    for addr in ranked_addresses:
        marker_heights = []
        marker_values = []

        for height in heights:
            if proposer_by_height.get(height, "") != addr:
                continue
```

```

            validators = validators_by_height.get(height, {})
            if addr not in validators:
                continue

            marker_heights.append(height)
            marker_values.append(validators[addr]["proposer_priority"])

        if marker_heights:
            plt.scatter(
                marker_heights,
                marker_values,
                marker="x",
                s=35,
                linewidths=1.2,
                color=line_color_by_addr.get(addr),
                label=None,
                zorder=5
            )

    plt.xlabel("Height")
    plt.ylabel("Proposer Priority")
    plt.title("Proposer Priority over Height")
    plt.grid(True)
    plt.legend(
        fontsize=6,
        loc="center left",
        bbox_to_anchor=(1.02, 0.5),
        borderaxespad=0.0,
        ncol=1
    )
    apply_axis_format(use_scientific, axis="both")
    plt.tight_layout(rect=[0, 0, 0.82, 1])
    plt.savefig(out_path, dpi=200, bbox_inches="tight")
    plt.close()


def print_summary(*__*, stats, per_height, diffs):
    print("===== Validator Analysis Summary =====")
    print(f"blocks loaded               : {len(blocks)}")
    print(f"unique validators           : {len(stats)}")

    if per_height:
        counts = [x["validator_count"] for x in per_height]
        totals = [x["total_voting_power"] for x in per_height]

        print(f"validator count min/max     : {min(counts)} / {max(counts)}")
        print(f"total voting power min/max  : {min(totals)} / {max(totals)}")

    added_total = sum(d["added_count"] for d in diffs)
    removed_total = sum(d["removed_count"] for d in diffs)
    power_changed_total = sum(d["power_changed_count"] for d in diffs)
    priority_changed_total = sum(d["priority_changed_count"] for d in diffs)

    print(f"total added events          : {added_total}")
    print(f"total removed events        : {removed_total}")
    print(f"total power changes         : {power_changed_total}")
```

```
    print(f"total priority changes      : {priority_changed_total}")
    print()

    print("Top validators by appearance_count")

    ranked = sorted(stats.items(), key=lambda x: (-x[1]["appearance_count"], x[0]))[:10]

    for i, (addr, st) in enumerate(ranked, 1):
        avg_vp = st["sum_voting_power"] / st["appearance_count"] if st["appearance_count
↪ "] else 0

        print(
            f"{i:2d}. {addr} "
            f"appear={st['appearance_count']} "
            f"first={st['first_height']} "
            f"last={st['last_height']} "
            f"avg_vp={avg_vp:.2f}"
        )


def main():
    os.makedirs(OUT_DIR, exist_ok=True)

    blocks = load_validator_files(INPUT_DIR)

    if not blocks:
        print(f"[ERROR] no valid JSON files found in {INPUT_DIR}")
        return 1

    stats = summarize_validator_presence(blocks)
    per_height = analyze_per_height(blocks)
    diffs = analyze_diffs(blocks)

    presence_csv = os.path.join(OUT_DIR, "validator_presence.csv")
    per_height_csv = os.path.join(OUT_DIR, "validator_set_per_height.csv")
    diff_summary_csv = os.path.join(OUT_DIR, "validator_diff_summary.csv")
    diff_details_json = os.path.join(OUT_DIR, "validator_diff_details.json")

    plot_validator_count_path = os.path.join(
        OUT_DIR,
        "validator_count_over_height.png"
    )

    plot_validator_changes_path = os.path.join(
        OUT_DIR,
        "validator_set_changes_over_height.png"
    )

    plot_priority_changed_count_path = os.path.join(
        OUT_DIR,
        "priority_changed_count_over_height.png"
    )

    plot_all_validators_voting_power_scientific_path = os.path.join(
        OUT_DIR,
        "all_validators_voting_power_scientific.png"
    )
```

```

    plot_all_validators_voting_power_plain_path = os.path.join(
        OUT_DIR,
        "all_validators_voting_power_plain.png"
    )

    plot_all_validators_proposer_priority_scientific_path = os.path.join(
        OUT_DIR,
        "all_validators_proposer_priority_scientific.png"
    )

    plot_all_validators_proposer_priority_plain_path = os.path.join(
        OUT_DIR,
        "all_validators_proposer_priority_plain.png"
    )

    write_presence_csv(stats, presence_csv)
    write_per_height_csv(per_height, per_height_csv)
    write_diff_summary_csv(diffs, diff_summary_csv)
    write_diff_details_json(diffs, diff_details_json)

    plot_validator_count(
        per_height,
        plot_validator_count_path,
        use_scientific=False
    )

    plot_validator_set_changes(
        diffs,
        plot_validator_changes_path,
        use_scientific=False
    )

    plot_priority_changed_count(
        diffs,
        plot_priority_changed_count_path,
        use_scientific=False
    )

    plot_all_validators_voting_power(
        blocks,
        stats,
        plot_all_validators_voting_power_scientific_path,
        use_scientific=True,
        fig_width=16,
        fig_height=8
    )

    plot_all_validators_voting_power(
        blocks,
        stats,
        plot_all_validators_voting_power_plain_path,
        use_scientific=False,
        fig_width=16,
        fig_height=8
    )

```

```
    plot_all_validators_proposer_priority(
        blocks,
        stats,
        plot_all_validators_proposer_priority_scientific_path,
        use_scientific=True,
        fig_width=16,
        fig_height=8
    )

    plot_all_validators_proposer_priority(
        blocks,
        stats,
        plot_all_validators_proposer_priority_plain_path,
        use_scientific=False,
        fig_width=PROPOSER_PRIORITY_PLAIN_FIG_WIDTH,
        fig_height=PROPOSER_PRIORITY_PLAIN_FIG_HEIGHT
    )

    print_summary(blocks, stats, per_height, diffs)

    print()
    print("===== Output Files =====")
    print(presence_csv)
    print(per_height_csv)
    print(diff_summary_csv)
    print(diff_details_json)
    print(plot_validator_count_path)
    print(plot_validator_changes_path)
    print(plot_priority_changed_count_path)
    print(plot_all_validators_voting_power_scientific_path)
    print(plot_all_validators_voting_power_plain_path)
    print(plot_all_validators_proposer_priority_scientific_path)
    print(plot_all_validators_proposer_priority_plain_path)

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

このスクリプトにより，ブロック高ごとのバリデータ数，Voting Power の合計，バリデータ集合の追加・削除，Voting Power や Proposer Priority の変化を分析できる．また，CSV および図として出力されるため，Proposer-Visualizer や表計算ソフトと組み合わせた分析にも利用できる．

Day1: 必須課題 1-3

ソースコード 1-3 を用いて，1-1 で取得したデータを分析，ソースコードの動作を説明し，バリデータ集合（Validator Set）の変化，Voting Power，Proposer Priority の推移について考察し，報告せよ．

ソースコード 20　分析スクリプト（1-4）

```
import os
import re
import *__*
import pandas as *__*
```

```
import matplotlib.pyplot as *__*

BLOCK_ANALYSIS_CSV = "blockheader_blockresults_analysis.csv"
VALIDATOR_PRESENCE_CSV = "out_o*__*_validator_analysis/validator_presence.csv"
VALIDATOR_SET_PER_HEIGHT_CSV = "out_*__*_validator_analysis/validator_set_per_height.csv"
RAW_BLOCK_JSON_DIR = "current_*__*"

OUT_DIR = "out_signature_block_interval_analysis"

MERGED_CSV = "signature_block_interval_validator_merged.csv"

SCATTER_SIMPLE = "sig_span_vs_block_diff_simple.png"
SCATTER_BY_VALIDATOR_COUNT = "sig_span_vs_block_diff_by_validator_count.png"
SCATTER_BY_TOTAL_VOTING_POWER = "sig_span_vs_block_diff_by_total_voting_power.png"
SCATTER_PROPOSER_RANK = "proposer_rank_vs_block_diff.png"
SCATTER_PREV_PROPOSER_RANK = "prev_proposer_rank_vs_block_diff.png"
CURRENT_PROPOSER_PREV_RANK_LINE = "current_proposer_prev_rank_over_height.png"
CURRENT_PROPOSER_PREV_RANK_SCATTER = "current_proposer_prev_rank_vs_block_diff.png"

MAX_SIGNATURE_SPAN_SEC = 60
MAX_BLOCK_DIFF_SEC = None


def ensure_columns(df: pd.DataFrame, required_columns: list[str], name: str):
    missing = [col for col in required_columns if col not in df.columns]
    if missing:
        raise ValueError(f"{name} に必要な列がありません: {missing}")


def extract_height_from_filename(filename: str):
    m = re.search(r"BlockNum_(\d+)\.json$", filename)
    if not m:
        return None
    return int(m.group(1))


def safe_int(value, default=0):
    try:
        return int(value)
    except Exception:
        return default


def load_data():
    block_df = pd.read_csv(BLOCK_ANALYSIS_CSV)
    presence_df = pd.read_csv(VALIDATOR_PRESENCE_CSV)
    per_height_df = pd.read_csv(VALIDATOR_SET_PER_HEIGHT_CSV)

    ensure_columns(
        block_df,
        ["height", "block_diff_sec", "sig_time_span_sec"],
        BLOCK_ANALYSIS_CSV
    )

    ensure_columns(
        per_height_df,
```

```
        ["height", "validator_count", "total_voting_power"],
        VALIDATOR_SET_PER_HEIGHT_CSV
    )

    ensure_columns(
        presence_df,
        ["address", "appearance_count"],
        VALIDATOR_PRESENCE_CSV
    )

    block_df["height"] = pd.to_numeric(block_df["height"], errors="coerce")
    block_df["block_diff_sec"] = pd.to_numeric(block_df["block_diff_sec"], errors="coerce")
    block_df["sig_time_span_sec"] = pd.to_numeric(block_df["sig_time_span_sec"], errors="coerce")

    per_height_df["height"] = pd.to_numeric(per_height_df["height"], errors="coerce")
    per_height_df["validator_count"] = pd.to_numeric(per_height_df["validator_count"], errors="coerce")
    per_height_df["total_voting_power"] = pd.to_numeric(per_height_df["total_voting_power"], errors="coerce")

    return block_df, presence_df, per_height_df


def load_raw_block_rows(raw_json_dir: str):
    rows = []

    if not os.path.isdir(raw_json_dir):
        print(f"[WARN] raw JSON directory not found: {raw_json_dir}")
        return rows

    for filename in sorted(os.listdir(raw_json_dir)):
        if not filename.endswith(".json"):
            continue

        height = extract_height_from_filename(filename)
        if height is None:
            continue

        path = os.path.join(raw_json_dir, filename)

        try:
            with open(path, "r", encoding="utf-8") as f:
                data = json.load(f)
        except Exception as e:
            print(f"[WARN] failed to read {path}: {e}")
            continue

        blockheader = data.get("blockheader", {}) or {}
        validators = data.get("validators", []) or []

        proposer_address = blockheader.get("proposer_address")

        normalized_validators = []

        for v in validators:
```

```
            if not isinstance(v, dict):
                continue

            address = v.get("address")
            if not address:
                continue

            normalized_validators.append({
                "address": address,
                "proposer_priority": safe_int(v.get("proposer_priority", 0)),
                "voting_power": safe_int(v.get("voting_power", 0)),
            })

        rows.append({
            "height": *__*,
            "proposer_address": proposer_*__*,
            "validators": normalized_validators,
            "validator_count_from_json": len(normalized_validators),
        })

    rows.sort(key=lambda x: x["height"])
    return rows


def calc_proposer_rank_for_block(proposer_address, validators):
    if not proposer_address or not validators:
        return {
            "proposer_priority_rank": None,
            "proposer_voting_power_rank": None,
            "proposer_priority_value": None,
            "proposer_voting_power": None,
        }

    sorted_by_priority = sorted(
        validators,
        key=lambda x: (-x["proposer_priority"], -x["voting_power"], x["address"])
    )

    sorted_by_voting_power = sorted(
        validators,
        key=lambda x: (-x["voting_power"], -x["proposer_priority"], x["address"])
    )

    proposer_priority_rank = None
    proposer_voting_power_rank = None
    proposer_priority_value = None
    proposer_voting_power = None

    for idx, row in enumerate(sorted_by_priority, start=1):
        if row["address"] == proposer_address:
            proposer_priority_rank = idx
            proposer_priority_value = row["proposer_priority"]
            proposer_voting_power = row["voting_power"]
            break

    for idx, row in enumerate(sorted_by_voting_power, start=1):
        if row["address"] == proposer_address:
```

```
            proposer_voting_power_rank = idx
            if proposer_voting_power is None:
                proposer_voting_power = row["voting_power"]
            if proposer_priority_value is None:
                proposer_priority_value = row["*__*"]
            break

    return {
        "proposer_priority_rank": *__*,
        "proposer_voting_power_rank": proposer_voting_power_rank,
        "proposer_priority_value": proposer_priority_value,
        "proposer_voting_power": proposer_voting_power,
    }


def calc_current_proposer_values_in_prev_block(proposer_address, prev_validators):
    if not proposer_address or not prev_validators:
        return {
            "current_proposer_priority_in_prev_block": None,
            "current_proposer_rank_in_prev_block": None,
            "current_proposer_voting_power_in_prev_block": None,
        }

    prev_sorted_by_priority = sorted(
        prev_validators,
        key=lambda x: (-x["proposer_priority"], -x["voting_power"], x["address"])
    )

    current_proposer_priority_in_prev_block = None
    current_proposer_rank_in_prev_block = None
    current_proposer_voting_power_in_prev_block = None

    for idx, row in enumerate(prev_sorted_by_priority, start=1):
        if row["address"] == proposer_address:
            current_proposer_priority_in_prev_block = row["proposer_priority"]
            current_proposer_rank_in_prev_block = idx
            current_proposer_voting_power_in_prev_block = row["voting_power"]
            break

    return {
        "current_proposer_priority_in_prev_block": current_proposer_priority_in_prev_block,
        "current_proposer_rank_in_prev_block": current_proposer_rank_in_prev_block,
        "current_proposer_voting_power_in_prev_block": current_proposer_voting_power_in_prev_block,
    }


def build_proposer_rank_df(raw_json_dir: str) -> pd.DataFrame:
    block_rows = load_raw_block_rows(raw_json_dir)

    if not block_rows:
        return pd.DataFrame()

    validators_by_height = {
        row["height"]: row["validators"]
        for row in block_rows
```

```
    }

    proposer_by_height = {
        row["height"]: row["proposer_address"]
        for row in block_rows
    }

    records = []

    for row in block_rows:
        height = row["height"]
        proposer_address = row["*__*"]
        validators = row["validators"]
        prev_height = height - 1

        proposer_rank_info = calc_proposer_rank_for_block(
            proposer_address,
            validators
        )

        current_proposer_prev_info = calc_current_proposer_values_in_prev_block(
            proposer_address,
            validators_by_height.get(prev_height, [])
        )

        prev_proposer_address = proposer_by_height.get(prev_height)
        prev_proposer_rank_info = calc_proposer_rank_for_block(
            prev_proposer_address,
            validators_by_height.get(prev_height, [])
        )

        records.append({
            "height": height,
            "proposer_address": proposer_address,
            "proposer_priority_rank": proposer_rank_info["proposer_priority_rank"],
            "proposer_voting_power_rank": proposer_rank_info["proposer_voting_power_rank"],
            "proposer_priority_value": proposer_rank_info["proposer_priority_value"],
            "proposer_voting_power": proposer_rank_info["proposer_voting_power"],
            "validator_count_from_json": row["validator_count_from_json"],
            "prev_height": prev_height,
            "prev_proposer_address": prev_proposer_address,
            "prev_proposer_priority_rank": prev_proposer_rank_info["proposer_priority_rank"],
            "prev_proposer_voting_power_rank": prev_proposer_rank_info["proposer_voting_power_rank"],
            "prev_proposer_priority_value": prev_proposer_rank_info["proposer_priority_value"],
            "prev_proposer_voting_power": prev_proposer_rank_info["proposer_voting_power"],
            "current_proposer_priority_in_prev_block": current_proposer_prev_info["current_proposer_priority_in_prev_block"],
            "current_proposer_rank_in_prev_block": current_proposer_prev_info["current_proposer_rank_in_prev_block"],
            "current_proposer_voting_power_in_prev_block": current_proposer_prev_info["current_proposer_voting_power_in_prev_block"],
        })
```

```python

    df = pd.DataFrame(records)

    numeric_cols = [
        "height",
        "proposer_priority_rank",
        "proposer_voting_power_rank",
        "proposer_priority_value",
        "proposer_voting_power",
        "validator_count_from_json",
        "prev_height",
        "prev_proposer_priority_rank",
        "prev_proposer_voting_power_rank",
        "prev_proposer_priority_value",
        "prev_proposer_voting_power",
        "current_proposer_priority_in_prev_block",
        "current_proposer_rank_in_prev_block",
        "current_proposer_voting_power_in_prev_block",
    ]

    for col in numeric_cols:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    df = df.sort_values("height").reset_index(drop=True)

    return df


def build_merged_df(
    block_df: pd.DataFrame,
    per_height_df: pd.DataFrame,
    proposer_rank_df: pd.DataFrame
) -> pd.DataFrame:
    use_block_cols = [
        "height",
        "timestamp",
        "block_diff_sec",
        "sig_time_span_sec",
        "signature_count",
        "num_transactions",
        "proposer_address",
    ]

    use_block_cols = [col for col in use_block_cols if col in block_df.columns]

    use_per_height_cols = [
        "height",
        "validator_count",
        "total_voting_power",
    ]

    merged = pd.merge(
        block_df[use_block_cols],
        per_height_df[use_per_height_cols],
        on="height",
        how="left"
```

```
    )

    if not proposer_rank_df.empty:
        proposer_use_cols = [
            "height",
            "proposer_priority_rank",
            "proposer_voting_power_rank",
            "proposer_priority_value",
            "proposer_voting_power",
            "validator_count_from_json",
            "prev_height",
            "prev_proposer_address",
            "prev_proposer_priority_rank",
            "prev_proposer_voting_power_rank",
            "prev_proposer_priority_value",
            "prev_proposer_voting_power",
            "current_proposer_priority_in_prev_block",
            "current_proposer_rank_in_prev_block",
            "current_proposer_voting_power_in_prev_block",
        ]

        proposer_use_cols = [
            col for col in proposer_use_cols
            if *__* in proposer_rank_df.columns
        ]

        merged = pd.merge(
            merged,
            proposer_rank_df[proposer_use_cols],
            on="height",
            how="left"
        )

    merged = merged.dropna(subset=["block_diff_sec", "sig_time_span_sec"])

    if MAX_SIGNATURE_SPAN_SEC is not None:
        merged = merged[merged["sig_time_span_sec"] <= MAX_SIGNATURE_SPAN_SEC]

    if MAX_BLOCK_DIFF_SEC is not None:
        merged = merged[merged["block_diff_sec"] <= MAX_BLOCK_DIFF_SEC]

    merged = merged.sort_values("height").reset_index(drop=True)

    return merged


def plot_simple_scatter(df: pd.DataFrame, out_path: str):
    if df.empty:
        print散布図を作成できません（": 有効なデータがありません。"）
        return

    plt.figure(figsize=(10, 7))
    plt.scatter(
        df["sig_time_span_sec"],
        df["block_diff_sec"],
        s=25,
        alpha=0.7
```

```
    )

    plt.xlabel("Validator Signature Time Span per Block (seconds)")
    plt.ylabel("Block Time Difference (seconds)")
    plt.title("Validator Signature Time Span vs Block Time Difference")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()

    print(f保存しました": {out_path}")


def plot_scatter_by_validator_count(df: pd.DataFrame, out_path: str):
    if df.empty:
        print("validator_count 散布図を作成できません: 有効なデータがありません。")
        return

    if "validator_count" not in df.columns or df["validator_count"].dropna().empty:
        print("validator_count がないため、散布図を作成できません。")
        return

    plt.figure(figsize=(10, 7))
    scatter = plt.scatter(
        df["sig_time_span_sec"],
        df["block_diff_sec"],
        c=df["validator_count"],
        s=25,
        alpha=0.75
    )

    plt.xlabel("Validator Signature Time Span per Block (seconds)")
    plt.ylabel("Block Time Difference (seconds)")
    plt.title("Validator Signature Time Span vs Block Time Difference")
    plt.grid(True)

    cbar = plt.colorbar(scatter)
    cbar.set_label("Validator Count")

    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()

    print(f保存しました": {out_path}")


def plot_scatter_by_total_voting_power(df: pd.DataFrame, out_path: str):
    if df.empty:
        *__*("total_voting_power 散布図を作成できません: 有効なデータがありません。")
        return

    if "total_voting_power" not in df.columns or df["total_voting_power"].dropna().empty
↪ :
        print("total_voting_power がないため、散布図を作成できません。")
        return

    plt.figure(figsize=(10, 7))
```

```
    scatter = plt.scatter(
        df["sig_time_span_sec"],
        df["block_diff_sec"],
        c=df["total_voting_power"],
        s=25,
        alpha=0.75
    )

    plt.xlabel("Validator Signature Time Span per Block (seconds)")
    plt.ylabel("Block Time Difference (seconds)")
    plt.*__*("Validator Signature Time Span vs Block Time Difference")
    plt.grid(True)

    cbar = plt.colorbar(scatter)
    cbar.set_label("Total Voting Power")

    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.*__*()

    print(f保存しました": {out_path}")


def plot_proposer_rank_vs_block_diff(df: pd.DataFrame, out_path: str):
    if df.empty:
        print("proposer rank 散布図を作成できません: 有効なデータがありません。")
        return

    if "proposer_priority_rank" not in df.columns:
        print("proposer_priority_rank がないため、散布図を作成できません。")
        return

    valid_df = df.dropna(subset=["proposer_priority_rank", "block_diff_sec"])

    if valid_df.empty:
        print("proposer_priority_rank と block_diff_sec の有効データがありません。")
        return

    plt.figure(figsize=(10, 7))
    plt.scatter(
        valid_df["proposer_priority_rank"],
        valid_df["block_diff_sec"],
        s=25,
        alpha=0.75
    )

    plt.xlabel("Block Proposer Rank")
    plt.ylabel("Block Time Difference (seconds)")
    plt.title("Block Proposer Rank vs Block Time Difference")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()

    print(f保存しました": {out_path}")


```

```
def plot_prev_proposer_rank_vs_block_diff(df: pd.DataFrame, out_path: str):
    if df.empty:
        print("previous proposer rank 散布図を作成できません: 有効なデータがありません。")
        return

    if "prev_proposer_priority_rank" not in df.columns:
        print("prev_proposer_priority_rank がないため、散布図を作成できません。")
        return

    valid_df = df.dropna(subset=["prev_proposer_priority_rank", "block_diff_sec"])

    if valid_df.empty:
        print("prev_proposer_priority_rank と block_diff_sec の有効データがありません。")
        return

    plt.figure(figsize=(10, 7))
    plt.scatter(
        valid_df["prev_proposer_priority_rank"],
        valid_df["block_diff_sec"],
        s=25,
        alpha=0.75
    )

    plt.xlabel("Previous Block Proposer Rank")
    plt.ylabel("Block Time Difference (seconds)")
    plt.title("Previous Block Proposer Rank vs Block Time Difference")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()

    print(f保存しました": {out_path}")


def plot_current_proposer_prev_rank_over_height(df: pd.DataFrame, out_path: str):
    if df.empty:
        print("current proposer previous rank の時系列図を作成できません: 有効なデータがありません。")
        return

    if "current_proposer_rank_in_prev_block" not in df.columns:
        print("current_proposer_rank_in_prev_block がないため、描画できません。")
        return

    valid_df = df.dropna(subset=[
        "height",
        "current_proposer_rank_in_prev_block",
    ])

    if valid_df.empty:
        print("current_proposer_rank_in_prev_block の有効データがありません。")
        return

    plt.figure(figsize=(12, 6))
    plt.plot(
        valid_df["height"],
        valid_df["current_proposer_rank_in_prev_block"],
```

```
        marker="o",
        markersize=3,
        linewidth=1
    )

    plt.xlabel("Height")
    plt.ylabel("Proposer Rank in Previous Block")
    plt.title("Proposer Rank in Previous Block over Height")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(out_path, dpi=200)
    plt.close()

    print(f保存しました": {out_path}")


def plot_current_proposer_prev_rank_vs_block_diff(df: pd.DataFrame, out_path: str):
    if df.empty:
        print("current proposer previous rank 散布図を作成できません: 有効なデータがありません。")
        return

    if "current_proposer_rank_in_prev_block" not in df.columns:
        print("current_proposer_rank_in_prev_block がないため、散布図を作成できません。")
        return

    valid_df = df.dropna(subset=[
        "current_proposer_rank_in_prev_block",
        "block_diff_sec",
    ])

    if valid_df.empty:
        print("current_proposer_rank_in_prev_block と block_diff_sec の有効データがありません。")
        return

    plt.figure(figsize=(10, 7))
    plt.scatter(
        valid_df["current_proposer_rank_in_prev_block"],
        valid_df["block_diff_sec"],
        s=25,
        alpha=0.75
    )

    plt.xlabel("Proposer Rank in Previous Block")
    plt.ylabel("Block Time Difference (seconds)")
    plt.title("Proposer Rank in Previous Block vs Block Time Difference")
    plt.grid(True)
    plt.tight_layout()
    plt.*__*(out_path, dpi=200)
    plt.close()

    print(f保存しました": {out_path}")


def calc_correlations(df: pd.DataFrame, x_col: str, y_col: str):
    valid_df = df.dropna(subset=[x_col, y_col])

```

```
    if len(valid_df) < 2:
        return None, None

    pearson = valid_df[x_col].corr(
        valid_df[y_col],
        method="pearson"
    )

    x_rank = valid_df[x_col].*__*()
    y_rank = valid_df[y_col].rank()

    spearman = x_rank.corr(
        y_rank,
        method="pearson"
    )

    return pearson, spearman


def print_corr(df: pd.DataFrame, x_col: str, y_col: str, title: str):
    if x_col not in df.columns or y_col not in df.columns:
        return

    pearson, spearman = calc_correlations(df, x_col, y_col)

    if pearson is not None and spearman is not None:
        print(f"\nCorrelation: {title}")
        print(f"  Pearson  : {pearson:.6f}")
        print(f"  Spearman : {spearman:.6f}")


def print_summary(df: pd.DataFrame, presence_df: pd.DataFrame):
    print("\n===== Signature Span vs Block Interval Summary =====")
    print(f"merged rows                  : {len(df)}")
    print(f"unique validators in presence: {presence_df['address'].nunique()}")

    if df.empty:
        print有効な分析データがありません。("")
        return

    print("\nValidator signature time span:")
    print(f"  min    : {df['sig_time_span_sec'].min():.6f} sec")
    print(f"  max    : {df['sig_time_span_sec'].max():.6f} sec")
    print(f"  mean   : {df['sig_time_span_sec'].mean():.6f} sec")
    print(f"  median : {df['sig_time_span_sec'].median():.6f} sec")
    print(f"  p90    : {df['sig_time_span_sec'].quantile(0.90):.6f} sec")
    print(f"  p99    : {df['sig_time_span_sec'].quantile(0.99):.6f} sec")

    print("\nBlock time difference:")
    print(f"  min    : {df['block_diff_sec'].min():.6f} sec")
    print(f"  max    : {df['block_diff_sec'].max():.6f} sec")
    print(f"  mean   : {df['block_diff_sec'].mean():.6f} sec")
    print(f"  median : {df['block_diff_sec'].median():.6f} sec")
    print(f"  p90    : {df['block_diff_sec'].quantile(0.90):.6f} sec")
    print(f"  p99    : {df['block_diff_sec'].quantile(0.99):.6f} sec")

    print_corr(
```

```
        df,
        "sig_time_span_sec",
        "block_diff_sec",
        "sig_time_span_sec vs block_diff_sec"
    )

    print_corr(
        df,
        "proposer_priority_rank",
        "block_diff_sec",
        "proposer_priority_rank vs block_diff_sec"
    )

    print_corr(
        df,
        "prev_proposer_priority_rank",
        "block_diff_sec",
        "prev_proposer_priority_rank vs block_diff_sec"
    )

    print_corr(
        df,
        "current_proposer_rank_in_prev_block",
        "block_diff_sec",
        "current_proposer_rank_in_prev_block vs block_diff_sec"
    )

    print_corr(
        df,
        "current_proposer_priority_in_prev_block",
        "block_diff_sec",
        "current_proposer_priority_in_prev_block vs block_diff_sec"
    )

    if "proposer_priority_rank" in df.columns:
        valid_rank_df = df.dropna(subset=["proposer_priority_rank"])

        if not valid_rank_df.empty:
            print("\nBlock proposer rank:")
            print(f"  min    : {valid_rank_df['proposer_priority_rank'].min():.0f}")
            print(f"  max    : {valid_rank_df['proposer_priority_rank'].max():.0f}")
            print(f"  mean   : {valid_rank_df['proposer_priority_rank'].mean():.2f}")
            print(f"  median : {valid_rank_df['proposer_priority_rank'].median():.2f}")

    if "prev_proposer_priority_rank" in df.columns:
        valid_prev_rank_df = df.dropna(subset=["prev_proposer_priority_rank"])

        if not valid_prev_rank_df.empty:
            print("\nPrevious block proposer rank:")
            print(f"  min    : {valid_prev_rank_df['prev_proposer_priority_rank'].min():.0f}")
            print(f"  max    : {valid_prev_rank_df['prev_proposer_priority_rank'].max():.0f}")
            print(f"  mean   : {valid_prev_rank_df['prev_proposer_priority_rank'].mean():.2f}")
            print(f"  median : {valid_prev_rank_df['prev_proposer_priority_rank'].median():.2f}")
```

```python

    if "current_proposer_rank_in_prev_block" in df.columns:
        valid_current_prev_rank_df = df.dropna(subset=["current_proposer_rank_in_prev_block"])

        if not valid_current_prev_rank_df.empty:
            print("\nCurrent proposer rank in previous block:")
            print(f"  min    : {valid_current_prev_rank_df['current_proposer_rank_in_prev_block'].min():.0f}")
            print(f"  max    : {valid_current_prev_rank_df['current_proposer_rank_in_prev_block'].max():.0f}")
            print(f"  mean   : {valid_current_prev_rank_df['current_proposer_rank_in_prev_block'].mean():.2f}")
            print(f"  median : {valid_current_prev_rank_df['current_proposer_rank_in_prev_block'].median():.2f}")

    if "current_proposer_priority_in_prev_block" in df.columns:
        valid_prev_priority_df = df.dropna(subset=["current_proposer_priority_in_prev_block"])

        if not valid_prev_priority_df.empty:
            print("\nCurrent proposer priority in previous block:")
            print(f"  min    : {valid_prev_priority_df['current_proposer_priority_in_prev_block'].min():.0f}")
            print(f"  max    : {valid_prev_priority_df['current_proposer_priority_in_prev_block'].max():.0f}")
            print(f"  mean   : {valid_prev_priority_df['current_proposer_priority_in_prev_block'].mean():.2f}")
            print(f"  median : {valid_prev_priority_df['current_proposer_priority_in_prev_block'].median():.2f}")

    if "validator_count" in df.columns:
        print("\nValidator count:")
        print(f"  min    : {df['validator_count'].min():.0f}")
        print(f"  max    : {df['validator_count'].max():.0f}")
        print(f"  mean   : {df['validator_count'].mean():.2f}")

    if "total_voting_power" in df.columns:
        print("\nTotal voting power:")
        print(f"  min    : {df['total_voting_power'].min():.0f}")
        print(f"  max    : {df['total_voting_power'].max():.0f}")
        print(f"  mean   : {df['total_voting_power'].mean():.2f}")


def main():
    os.makedirs(OUT_DIR, exist_ok=True)

    block_df, presence_df, per_height_df = load_data()
    proposer_rank_df = build_proposer_rank_df(RAW_BLOCK_JSON_DIR)

    merged_df = build_merged_df(
        block_df,
        per_height_df,
        proposer_rank_df
    )

    merged_csv_path = os.path.join(OUT_DIR, MERGED_CSV)
```

```

    merged_df.to_csv(
        merged_csv_path,
        index=False,
        encoding="utf-8-sig"
    )

    print(f保存しました": {merged_csv_path}")

    simple_path = os.path.join(OUT_DIR, SCATTER_SIMPLE)
    validator_count_path = os.path.join(OUT_DIR, SCATTER_BY_VALIDATOR_COUNT)
    total_voting_power_path = os.path.join(OUT_DIR, SCATTER_BY_TOTAL_VOTING_POWER)
    proposer_rank_path = os.path.join(OUT_DIR, SCATTER_PROPOSER_RANK)
    prev_proposer_rank_path = os.path.join(OUT_DIR, SCATTER_PREV_PROPOSER_RANK)
    current_proposer_prev_rank_line_path = os.path.join(
        OUT_DIR,
        CURRENT_PROPOSER_PREV_RANK_LINE
    )
    current_proposer_prev_rank_scatter_path = os.path.join(
        OUT_DIR,
        CURRENT_PROPOSER_PREV_RANK_SCATTER
    )

    plot_simple_scatter(
        merged_df,
        simple_path
    )

    plot_scatter_by_validator_count(
        merged_df,
        validator_count_path
    )

    plot_scatter_by_total_voting_power(
        merged_df,
        total_voting_power_path
    )

    plot_proposer_rank_vs_block_diff(
        merged_df,
        proposer_rank_path
    )

    plot_prev_proposer_rank_vs_block_diff(
        merged_df,
        prev_proposer_rank_path
    )

    plot_current_proposer_prev_rank_over_height(
        merged_df,
        current_proposer_prev_rank_line_path
    )

    plot_current_proposer_prev_rank_vs_block_diff(
        merged_df,
        current_proposer_prev_rank_scatter_path
    )
```

```

    print_summary(merged_df, presence_df)

    print("\n===== Output Files =====")
    print(merged_csv_path)
    print(simple_path)
    print(validator_count_path)
    print(total_voting_power_path)
    print(proposer_rank_path)
    print(prev_proposer_rank_path)
    print(current_proposer_prev_rank_line_path)
    print(current_proposer_prev_rank_scatter_path)

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

このスクリプトにより，ブロック高ごとのバリデータ数，Voting Power の合計，バリデータ集合の追加・削除，Voting Power や Proposer Priority の変化を分析できる．また，CSV および図として出力されるため，Proposer-Visualizer や表計算ソフトと組み合わせた分析にも利用できる．

Day1: 必須課題 1-4

ソースコード 1-4 を用いて，1-1,1-2,1-3 で取得したデータも用いて分析し，ソースコードの動作を説明し，Block Proposer Rank とブロック生成時間間隔の関係性を確認する散布図，Validator 署名時間とブロック生成時間間隔の関係性を確認する散布図などを描画し，関係性についてについて考察し，報告せよ．

3.6 Proposer-Visualizer

Proposer-Visualizer は，Cosmos / CometBFT 系ブロックチェーンから取得したブロックデータおよびバリデータ関連データを可視化するための Web アプリケーションである．本ツールの目的は，ブロック提案者（Proposer）の出現傾向，ブロック間隔，バリデータ（Validator）の優先度推移などを視覚的に確認し，チェーン上での挙動を理解しやすくすることである．

このツールでは，取得済みの CSV や JSON を入力として，ブラウザ上で複数の可視化結果を表示する．

共有された実行ファイル群および必要なデータを所定の場所に配置し，指定された手順に従ってアプリケーションを起動することで，同一の可視化結果を確認できる．

起動コマンド例をコマンド 21 に示す．

コマンド 21　Proposer-Visualizer の起動コマンド例

```
bash start.sh
```

本実験では，前段で取得したブロックデータおよび分析結果を Proposer-Visualizer に読み込ませ，ブロック提案者の出現傾向，ブロック間隔の特徴，バリデータ情報との関係などを確認する．これにより，ブロックチェーンの合意形成過程を定性的かつ定量的に理解することを目的とする．

練習課題

1-5_visualizer-convert.py を実行し，Proposer-Visualizer 用データを作成した後，Proposer-Visualizer を起動し，分析結果ファイルを読み込ませ Visualizer が起動することを確認せよ．

Day1: 必須課題 1-5

Proposer-Visualizer を起動し，分析結果ファイルを読み込ませたうえで，Validator 遷移および Proposer の出現傾向について考察し，報告せよ．共同実験者の分析結果を読み込ませた結果と比較，考察し，報告せよ．また 1 − 5 のソースコードの動作を説明すること．

3.7 CW20 の実装

本実験で実装対象とするトークンは，CosmWasm におけるファンジブルトークン標準である **CW20** である．CW20 は，Ethereum における ERC-20 に対応する概念を CosmWasm 向けに定義したものであり，トークンの発行，残高管理，送金，許可（Allowance）などの基本機能を備える．

CW20 コントラクトでは，一般に初期化時にトークン名，シンボル，decimal 桁数，初期残高，および mint 権限の有無などを設定する．その後，ユーザはコントラクトに対して送金や残高照会などの操作を行うことで，ブロックチェーン上でトークンを扱うことができる．

3.8 Injective を用いた独自トークンの作成

本節では，Injective の `tokenfactory` モジュールを用いて，ネイティブ denom 形式の独自トークンを作成する方法を扱う．前節で扱った CW20 はスマートコントラクトとしてトークン状態を管理する方式であるのに対し，Token Factory ではチェーンの標準機能としてトークン denom を作成する．

3.8.1 事前確認

まず，鍵名 `mykey` のウォレットを作成する．ウォレット作成コマンドをコマンド 22 に示す．

コマンド 22　ウォレット作成コマンド

```
injectived keys add mykey
```

次に，testnet 用の共通設定を環境変数として与える．環境変数の設定コマンドをコマンド 23 に示す．

コマンド 23　Injective testnet 用環境変数の設定コマンド

```
export CHAIN_ID=injective-888
export NODE=https://injective-testnet-rpc.publicnode.com:443
export GAS_PRICES=500000000inj
export GAS=1000000
export KEY_NAME=mykey
export MY_ADDR=$(injectived keys show $KEY_NAME -a)
export SUBDENOM=mytoken
```

`https://cloud.google.com/application/web3/faucet/injective/testnet`

faucet 実行後に，残高が反映されたかを確認する．残高確認コマンドをコマンド 24 に示す．

コマンド 24　faucet 実行後の残高確認コマンド

```
injectived query bank balances $MY_ADDR --node=$NODE
```

3.8.2　独自 denom の作成

Token Factory を用いて独自 denom を作成する．独自 denom の作成コマンドをコマンド 25 に示す．

コマンド 25　独自 denom の作成コマンド

```
injectived tx tokenfactory create-denom \
  $SUBDENOM "My Token" "MTK" 6 \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=$GAS
```

作成後，denom は次の形式となる．

ソースコード 26　Token Factory で作成される denom の形式例

```
factory/${MY_ADDR}/${SUBDENOM}
```

3.8.3　denom metadata の設定

denom metadata の設定コマンドをコマンド 27 に示す．

コマンド 27　denom metadata の設定コマンド

```
injectived tx tokenfactory set-denom-metadata \
  "My Token Description" \
  "$DENOM" \
  "MTK" \
  "My Token" \
  "MTK" \
  "" \
  "" \
  "[{\"denom\":\"$DENOM\",\"exponent\":0,\"aliases\":[]},{\"denom\":\"MTK\",\"exponent
↪ \":6,\"aliases\":[]}]" \
  6 \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=$GAS
```

ここでは base denom を `$DENOM`，表示用 denom を `MTK` とし，1 `MTK` $= 10^6$ base unit となるように設定している．

3.8.4　トークンの mint

mint コマンドをコマンド 28 に示す．

コマンド 28　トークンの mint コマンド

```
injectived tx tokenfactory mint 1000000${DENOM} \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=$GAS
```

3.8.5 残高確認

mint 後に，自アカウントの残高を確認する．残高確認コマンドをコマンド 29 に示す．

コマンド 29　mint 後の残高確認コマンド

```
injectived query bank balances $MY_ADDR --node=$NODE
```

3.8.6 INJ の送金確認

独自トークンや CW20 トークンの送金実験に進む前に，Injective testnet 上で通常の INJ 送金を実行し，トランザクションが正しく処理されることを確認する．受信先アドレスを RECIPIENT とすると，送金コマンドはコマンド 30 のようになる．

コマンド 30　INJ の送金確認コマンド

```
export RECIPIENT=inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

injectived tx bank send \
  $MY_ADDR \
  $RECIPIENT \
  1000000000000000000inj \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=$GAS
```

送金後は，トランザクションハッシュをブロックチェーンエクスプローラーで確認し，送金がブロックに取り込まれたことを確認する．

Day1: 必須課題 2-1

INJ を他の共同実験者間で送金し，送金結果をブロックチェーンエクスプローラーで確認し報告せよ．また，TX Hash と送金完了までに要した時間，10 回以上の平均時間をまとめ報告せよ．

3.8.7 Token Factory による独自トークンの送金

作成した独自トークンがネイティブ資産として扱われることを確認するため，別アカウントへ送金する．

コマンド 31　独自トークンの送金コマンド

```
export RECIPIENT=inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

injectived tx bank send \
  $MY_ADDR \
```

```
  $RECIPIENT \
  1000000${DENOM} \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=$GAS
```

Day1: 必須課題 2-2

Token factory で作成した独自トークンを他の共同実験者間で送金し，送金結果をブロックチェーンエクスプローラーで確認し報告せよ．また，TX Hash と送金完了までに要した時間，10 回以上の平均時間をまとめ報告せよ．

3.8.8 トークンの burn

作成者 admin は，自身の token を burn することもできる．

コマンド 32　トークンの burn コマンド

```
injectived tx tokenfactory burn 1000000${DENOM} \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=$GAS
```

3.9 CW20 コントラクトを用いた独自トークンの作成

前節では，Injective の `tokenfactory` モジュールを用いた独自トークンの作成方法を扱った．これに対し，本節では CosmWasm のスマートコントラクトとして CW20 を利用し，そのコントラクトを用いて独自トークンを作成する方法を扱う．本実験では，独自に CW20 コントラクトを一から実装するのではなく，CosmWasm の `cw-plus` リポジトリに含まれる `cw20-base` をサンプルコードとして利用する [10].

3.9.1 サンプルコードの取得

コマンド 33　CW20 サンプルコードの取得コマンド

```
git clone https://github.com/CosmWasm/cw-plus.git
cd cw-plus/contracts/cw20-base
```

3.9.2 サンプルコードのビルド

コマンド 34　CW20 サンプルコードのビルドコマンド

```
cargo wasm
cp ../../target/wasm32-unknown-unknown/release/cw20_base.wasm .
```

3.9.3 コントラクトコードの保存

コマンド 35　CW20 コントラクト保存コマンド例

```
injectived tx wasm store cw20_base.wasm \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=3000000
```

3.9.4 初期化メッセージの作成

ソースコード 36　CW20 コントラクトの初期化メッセージ例

```
{
  "name": "My CW20 Token",
  "symbol": "MCW",
  "decimals": 6,
  "initial_balances": [
    {
      "address": "inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
      "amount": "1000000"
    }
  ],
  "mint": {
    "minter": "inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "cap": null
  },
  "marketing": null
}
```

3.9.5 コントラクトのインスタンス化

コマンド 37　CW20 コントラクトのインスタンス化コマンド例

```
injectived tx wasm instantiate <CODE_ID> cw20_init.json \
  --label "my-cw20-token" \
  --no-admin \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=2000000
```

3.9.6 残高照会

コマンド 38　CW20 残高照会コマンド例

```
injectived query wasm contract-state smart <CONTRACT_ADDRESS> \
  '{"balance":{"address":"inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"}}' \
  --node=$NODE
```

3.9.7 送金処理

CW20 の送金は，コントラクトに対して `transfer` メッセージを送ることで実行する．

コマンド 39　CW20 の送金コマンド例

```
injectived tx wasm execute <CONTRACT_ADDRESS> \
  '{"transfer":{"recipient":"inj1yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy","amount":"500000"}}' \
  --from=$KEY_NAME \
  --chain-id=$CHAIN_ID \
  --node=$NODE \
  --gas-prices=$GAS_PRICES \
  --gas=300000
```

3.9.8 8. mint および burn

mint 権限を持つアドレスが設定されている場合には，追加発行を行うこともできる．

ソースコード 40　CW20 の mint メッセージ例

```
{
  "mint": {
    "recipient": "inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
    "amount": "1000000"
  }
}
```

Day1: 必須課題 2-3

CW-20 のスマートコントラクトで作成した独自トークンを他の共同実験者間で送金し，送金結果をブロックチェーンエクスプローラーで確認し報告せよ．また，TX Hash と送金完了までに要した時間，10 回以上の平均時間，他の共同実験者から受け取った CW20 トークンが，自身のウォレット上で確認できたことをまとめ報告せよ．

Day1: 必須課題 2-4

INJ，Token Factory による独自トークン，CW20 トークンの送金時間を比較し，それぞれの時間特性について考察せよ．

4 Day2: ブロックチェーンのセキュリティについて（改ざん耐性の確認）

編集中

5 Day3

編集中

参考文献

[1] Jae Kwon and Ethan Buchman, *Cosmos: A Network of Distributed Ledgers*, Cosmos Whitepaper. Available: https://github.com/cosmos/cosmos/raw/master/whitepaper/cosmos-sdk-intro.pdf Accessed: 2026-04-21.

[2] Cosmos Labs, "What is the Cosmos SDK," Cosmos SDK Documentation. Available: https://docs.cosmos.network/sdk/v0.53/learn/intro/overview Accessed: 2026-04-21.

[3] Cosmos Labs, "IBC-Go Documentation: Introduction," Cosmos Documentation. Available: https://docs.cosmos.network/ibc/latest/intro Accessed: 2026-04-21.

[4] Cosmos IBC Team, "Inter-Blockchain Communication Protocol (IBC)," cosmos/ibc repository. Available: https://github.com/cosmos/ibc Accessed: 2026-04-21.

[5] CometBFT Team, "CometBFT RPC Specification," CometBFT Documentation. Available: https://docs.cometbft.com/main/spec/rpc/ Accessed: 2026-04-21.

[6] CometBFT Team, "Validators," CometBFT Documentation. Available: https://docs.cometbft.com/main/core/validators Accessed: 2026-04-21.

[7] Injective Labs, "Creating Tokens," Injective Docs. Available: https://docs.injective.network/developers/assets/token-create Accessed: 2026-04-21.

[8] Injective Labs, "Launch a Token," Injective Docs. Available: https://docs.injective.network/developers-defi/token-launch Accessed: 2026-04-21.

[9] Injective Labs, "Commands," Injective Docs. Available: https://docs.injective.network/developers/injectived/advanced Accessed: 2026-04-21.

[10] CosmWasm Team, "cw20-base," in *cw-plus*. Available: https://github.com/CosmWasm/cw-plus/tree/main/contracts/cw20-base Accessed: 2026-04-21.

[11] Injective Labs, "Your First CosmWasm Smart Contract," Injective Docs. Available: https://docs.injective.network/developers-cosmwasm/smart-contracts/your-first-smart-contract Accessed: 2026-04-21.

[12] FUJITA TAKUYA, "Cosmo SDK ベースのチェーン『Neutron』でのトークン発行をしてみよう！", STIR LAB. Available: https://lab.stir.network/neutron-token-mint/ Accessed: 2026-04-21.

[13] SimBlock Project, "SimBlock," Available: https://github.com/dsg-titech/simblock Accessed: 2026-04-21.

[14] SimBlock Project, "SimBlock Visualizer," Available: https://dsg-titech.github.io/simblock-visualizer/ Accessed: 2026-04-21.

[15] Wikipedia, "Wide Area Network," Available: https://ja.wikipedia.org/wiki/Wide_Area_Network Accessed: 2026-04-21.

付録 A　初学者のための用語集

本付録では，本実験で扱う主要な用語を整理する．ブロックチェーン (Blockchain)，Cosmos，CometBFT，Injective，CW20，SimBlock などの内容を理解するためには，各用語の意味をあらかじめ把握しておくことが重要である．以下の用語は実験中に繰り返し登場するため，必要に応じて参照すること．

A.1　ブロックチェーンに関する基本用語

ブロックチェーン（Blockchain）　取引データや状態更新の情報をブロック単位で記録し，それらを時系列に連結して管理する分散型台帳である．一度記録されたデータを改ざんするには，後続のブロックも含めて変更する必要があるため，改ざん耐性を持つ構造として利用される．

ブロック（Block）　ブロックチェーンにおける記録単位である．ブロックには，ブロック高，生成時刻，提案者，トランザクション，署名情報などが含まれる．

ブロック高（Block Height）　ブロックチェーン上で各ブロックに付与される番号である．一般に，チェーンの先頭から何番目のブロックであるかを表す．たとえば，ブロック高 1000 は，そのチェーンにおける 1000 番目のブロックを意味する．

トランザクション（Transaction）　ブロックチェーン上で実行される操作や取引のことである．送金，スマートコントラクト実行，トークン発行，投票などがトランザクションとして記録される．

アドレス（Address）　ブロックチェーン上のアカウントを識別するための文字列である．送金元，送金先，コントラクト管理者，バリデータなどを識別するために用いられる．

残高（Balance）　あるアドレスが保有しているトークン量である．本実験では，`injectived query bank balances` などのコマンドにより確認する．

手数料（Fee）　トランザクションをブロックチェーンに送信する際に支払う費用である．手数料は，トランザクション処理やネットワーク利用に対する対価として扱われる．

ファイナリティ（Finality）　あるトランザクションやブロックが確定し，原則として取り消されない状態になる性質である．ファイナリティが短いほど，利用者は早く取引確定を確認できる．

スマートコントラクト（Smart Contract）　ブロックチェーン上で実行されるプログラムである．あらかじめ定義された条件や処理に従って，トークン発行，送金，状態管理などを自動的に実行する．

A.2　Cosmos / CometBFT に関する用語

Cosmos　複数の独立したブロックチェーンを相互接続することを目的としたブロックチェーンエコシステムである．「ブロックチェーンのインターネット」という考え方に基づき，目的ごとに設計されたチェーン同士を連携させることを重視している．

Cosmos SDK　Cosmos 系ブロックチェーンを構築するための開発フレームワークである．送金，アカウント管理，ステーキング，ガバナンスなどの機能をモジュールとして利用できる．

CometBFT　Cosmos 系チェーンで用いられる BFT 系コンセンサスエンジンである．旧 Tendermint Core の後継として扱われ，ブロック生成や合意形成を担う．

Tendermint BFT　Byzantine Fault Tolerance を前提としたコンセンサス方式である．一部のノードが故障

または不正な挙動をしても，ネットワーク全体として正しい合意を得ることを目的とする．

IBC（Inter-Blockchain Communication） 異なるブロックチェーン間でトークンやメッセージをやり取りするための通信プロトコルである．Cosmos における相互運用性を支える中核技術である．

ブロックチェーンのインターネット（Internet of Blockchains） 複数の独立したブロックチェーンを相互接続し，全体として一つの大きなネットワークを構成するという考え方である．インターネットが複数のネットワークを接続して成り立つのと同様に，Cosmos では IBC により複数チェーンを接続する．

RPC **ノード**（RPC Node） Remote Procedure Call node の略であり，外部プログラムからブロックチェーンの情報を問い合わせたり，トランザクションを送信したりするための接続先である．本実験では，ブロック情報やバリデータ情報の取得に RPC エンドポイントを利用する．

RPC **エンドポイント**（RPC Endpoint） RPC ノードへアクセスするための URL である．たとえば，`/block`，`/validators`，`/status` などのエンドポイントを通じて，ブロック情報やバリデータ情報を取得する．

チェーン ID（Chain ID） ブロックチェーンネットワークを識別するための ID である．トランザクション送信時には，誤ったネットワークへ送信しないようにチェーン ID を指定する．

バリデータ（Validator） ブロックチェーンの合意形成に参加するノードである．Cosmos / CometBFT 系チェーンでは，バリデータがブロック提案や署名を行い，チェーンの安全性と継続性を支える．

バリデータ集合（Validator Set） あるブロック高において合意形成に参加するバリデータの集合である．本実験では，各ブロック高における Validator Set を取得し，バリデータ数，投票力（Voting Power），提案者優先度（Proposer Priority）などを分析する．

投票力（Voting Power） バリデータが合意形成において持つ投票力である．Voting Power が大きいほど，合意形成における影響力も大きくなる．

提案者（Proposer） あるブロックを提案するバリデータである．各ブロックでは，バリデータ集合の中から提案者が選ばれ，その提案者がブロックを生成する．

提案者アドレス（Proposer Address） あるブロックを提案したバリデータのアドレスである．ブロックヘッダに含まれ，ブロック生成者を識別するために用いる．

提案者優先度（Proposer Priority） CometBFT 系の提案者選出に関係する優先度である．本実験では，前ブロックの Proposer Priority と次ブロックの Proposer の関係を分析対象とする．

署名（Signature） バリデータがブロックに対する合意を示すために付与する暗号学的な証明である．本実験では，ブロック内の署名数を確認することで，合意形成に関わった情報を分析する．

A.3 Injective / CosmWasm / CW20 に関する用語

Injective Cosmos SDK を基盤とするブロックチェーンの一つである．本実験では，Injective testnet を用いて CLI 操作，トークン作成，スマートコントラクト操作を確認する．

injectived Injective チェーンを操作するための CLI 兼ノードデーモンである．鍵管理，残高確認，トランザクション送信，CosmWasm コントラクト操作などに用いる．

Testnet 実験や開発のために用意されたテスト用ネットワークである．本実験では，Injective の testnet である `injective-888` を利用する．

Faucet Testnet 用のトークンを取得するための仕組みである．実験では，Faucet により取得した testnet 用

INJ を用いて手数料支払いやトランザクション実行を行う．

Token Factory　Cosmos SDK 系チェーンにおいて，ネイティブ denom を作成するための機能である．Injective では，`tokenfactory` モジュールを用いて独自トークンを作成できる．

denom　トークンの単位や識別名を表す文字列である．Token Factory で作成される denom は，`factory/{creator address}/{subdenom}` の形式をとる．

subdenom　Token Factory で作成する独自トークンの名前部分である．作成者アドレスと組み合わさることで，最終的な denom が決定される．

ネイティブトークン（Native Token）　チェーンの標準機能として扱われるトークンである．Token Factory によって作成された denom は，銀行モジュールなどを通じてネイティブ資産として扱われる．

CosmWasm　Cosmos SDK 系チェーン上でスマートコントラクトを実行するための仕組みである．Rust などで記述したコントラクトを WASM 形式にビルドし，チェーン上に保存・インスタンス化して利用する．

WASM（WebAssembly）　WebAssembly の略であり，スマートコントラクトを実行可能な形式にコンパイルしたものである．CosmWasm では，WASM ファイルをチェーンに保存してコントラクトとして利用する．

CW20　CosmWasm におけるファンジブルトークン標準である．Ethereum の ERC-20 に相当する概念であり，トークンの送金，残高確認，mint，burn，allowance 管理などを扱う．

cw20-base　CosmWasm の `cw-plus` リポジトリに含まれる CW20 の基本実装である．本実験では，CW20 コントラクトのサンプルコードとして利用する．

コード ID（Code ID）　チェーン上に保存された WASM コントラクトコードを識別する番号である．コントラクトをインスタンス化する際に指定する．

コントラクトアドレス（Contract Address）　インスタンス化されたスマートコントラクトのアドレスである．残高照会，送金，mint，burn などの操作では，このコントラクトアドレスを指定する．

初期化（Instantiate）　チェーン上に保存されたコントラクトコードから，実際に利用可能なコントラクトインスタンスを作成する操作である．初期化メッセージにより，トークン名や初期残高などを設定する．

実行（Execute）　スマートコントラクトに対して状態変更を伴う処理を実行する操作である．CW20 では，送金，mint，burn などが Execute に該当する．

照会（Query）　スマートコントラクトの状態を読み取る操作である．CW20 では，残高照会やトークン情報の確認などが Query に該当する．

発行（Mint）　新たにトークンを発行する操作である．CW20 では，mint 権限を持つアドレスのみが追加発行を行える．

焼却（Burn）　保有しているトークンを焼却し，供給量を減らす操作である．

許可量（Allowance）　あるアドレスに対し，自分のトークンを一定量まで利用する権限を与える仕組みである．CW20 では，`IncreaseAllowance` や `DecreaseAllowance` により管理される．

初期残高（Initial Balance）　コントラクトの初期化時に特定のアドレスへ割り当てるトークン量である．CW20 では，`initial_balances` によって設定する．

小数桁数（Decimals）　トークンの最小単位と表示単位の関係を表す数値である．たとえば decimals が 6 の場合，表示上の 1 トークンは内部単位の 10^6 に相当する．

A.4 データ分析・可視化に関する用語

JSON（JavaScript Object Notation） 構造化データを保存・交換するための形式である．本実験では，ブロック情報とバリデータ情報を JSON ファイルとして保存する．

CSV（Comma-Separated Values） 表形式のデータをテキストで保存する形式である．本実験では，分析スクリプトにより生成された結果を CSV として保存する．

pandas Python のデータ分析用ライブラリである．CSV 出力や表形式データの処理に用いる．

matplotlib Python のグラフ描画ライブラリである．バリデータ数や Voting Power の推移などを可視化するために用いる．

Proposer-Visualizer ブロック提案者，ブロック間隔，バリデータ情報などを可視化するための Web アプリケーションである．取得・分析済みの CSV や JSON を読み込むことで，チェーンの挙動を視覚的に確認できる．

バリデータ遷移（Validator Transition） ブロック高の変化に伴って，バリデータ集合，Voting Power，Proposer Priority などがどのように変化するかを表す．本実験では，Validator Transition を可視化・分析することで，合意形成に関わるノード群の変化を理解する．

時系列データ（Time-Series Data） 時間やブロック高の順序に沿って並べられたデータである．本実験では，ブロック高ごとのバリデータ数や Voting Power の変化を時系列データとして扱う．

ヒストグラム（Histogram） データの分布を棒グラフとして表す可視化手法である．本実験では，ブロック間隔の分布などを確認するために用いる．

A.5 SimBlock に関する用語

SimBlock ブロックチェーンネットワークの挙動を模擬するシミュレーションソフトウェアである．ノード数，通信遅延，ブロック生成間隔などの条件を変更し，ブロック伝播や分岐の発生を観察できる．

ブロック伝播（Block Propagation） あるノードで生成されたブロックが，他のノードへ送信され，ネットワーク全体に共有されていく過程である．

ブロック分岐（Fork） 複数のノードがほぼ同時にブロックを生成した結果，一時的に複数の候補チェーンが存在する状態である．

孤立ブロック（Orphan Block） 分岐の結果，最終的に正規チェーンに採用されなかったブロックである．

ネットワーク遅延（Network Latency） あるノードから別のノードへデータが届くまでにかかる時間である．ネットワーク遅延が大きいほど，ブロック分岐が発生しやすくなる．

ブロック生成間隔（Block Generation Interval） 連続するブロックが生成される時間間隔である．ブロック生成間隔が短く，ブロック伝播時間が長い場合，分岐が起こりやすくなる．

帯域幅（Bandwidth） ネットワーク上で単位時間あたりに送受信できるデータ量である．帯域幅が小さい場合，ブロック伝播に時間がかかり，分岐発生に影響する可能性がある．

ノード接続数（Node Degree） あるノードが接続している他ノードの数である．接続数は，ブロックやトランザクションの伝播速度に影響する．